Today
Welcome to your next lesson: Knowing when "done" means done.

I'm preparing your personalized lesson based on what you've learned so far, what's coming up next, and the best ways for you to really learn it.

Step by step, you're moving forward.

Imagine a friend asks you to write a Python function temperature_to_label(temp) that returns "cold", "warm", or "hot" based on a number.

The real question is: how do you know when you're actually done with this function? Not "it works for a couple of tests," but done-done ☑.

In this lesson, we'll answer that by:

Finding edge cases and boundaries
Turning them into clear pass/fail checks (acceptance criteria)

To start simple:

If you hear "function that labels temperatures as cold / warm / hot," what's your first instinct about edge cases — do you think more about:

A) Very low / very high numbers

B) Non-number inputs like "hot" or None

Just type A or B, no explanation needed.

b

Nice, B is a great instinct 👍 — "what if they pass something weird?" is exactly the kind of thinking we want.

Let's nail one idea at a time: boundary conditions.

Boundary condition = an input that is right on the edge of a rule, but still a valid input.

Example with temperature_to_label(temp):

Say we decide:
temp < 10 → "cold"
10 <= temp < 25 → "warm"
temp >= 25 → "hot"

Then the boundary conditions are the "border" numbers:

9, 10, 25, 24
(because they sit right around the cutoffs 10 and 25)

These are still valid numbers, just exactly where bugs like to hide.

Does that picture of "border numbers around 10 and 25" as boundary conditions make sense

so far, or should I walk through one tiny example with expected outputs?

Makes sense so far

What do you think — which one is the boundary case here? 👇

Which of these is a boundary condition for this rule?

temp < 10 → "cold"
10 <= temp < 25 → "warm"
temp >= 25 → "hot"
A.

temp = 37

B.

temp = "hot"

C.

temp = 10

D.

temp = None

🎯 Yes, exactly — temp = 10 is the key boundary here.

Quick reason why:

10 is right on the border between "cold" and "warm":
Below 10 → "cold"
From 10 up to (but not including) 25 → "warm"

So if there's a bug, it often shows up right at 10 or 25.

Next small step: let's contrast this with invalid edge cases (like None or "hot" as input) so they don't get mixed up.

Before I explain: when you see temp = "hot" or temp = None, do you personally think:

"Still valid, just tricky"
or "That's invalid input, the function shouldn't really accept that"?

You can just answer: valid or invalid.

invalid

Nice instinct — I agree: those are invalid inputs ☑

So now we can draw a clean line:

Boundary condition → valid input, but on the edge
e.g. temp = 10, temp = 25
Invalid edge case → input that breaks the "contract"
e.g. temp = "hot", temp = None

Both are "special cases," but we treat them differently when we define what "done" means.

Next, I want to show how we turn this into a clear test (acceptance criterion) like:

If temp = 10, the function must return "warm".

Want to try a tiny check on boundary vs invalid before we do that, or go straight to acceptance criteria?

Just say: quiz or move on.

Quiz

What do you think? Pick the one that’s a valid “border” number, not a broken input 👇

Which of these is a boundary condition, not an invalid input, for our temperature rules?

Rules:

temp < 10 → "cold"
10 <= temp < 25 → "warm"
temp >= 25 → "hot"
A.

temp = 25

B.

temp = None

C.

temp = "15" (string)

D.

temp = "hot"

Nice work again 🎯

You picked temp = 25, which is spot on:

It’s a number → valid input
It sits right on the border between "warm" and "hot" (because temp >= 25 becomes "hot")

So that’s exactly the kind of value we want to test carefully.

Now, one new idea: acceptance criteria.

An acceptance criterion is just a clear pass/fail statement like:

If temp = 10, the function returns "warm".

No maybes, no “should kind of work” — either it passes or it fails.

Your turn:

For our temperature rules, if temp = 9, what should the function return? Just type the word you think it should return.

cold

Perfect ✸

You just wrote, in your head, an acceptance criterion:

If temp = 9, the function must return "cold".

That’s exactly the style we want: specific input → specific expected output → pass/fail.

Let’s build one more together, then I’ll have you try one on your own.

Using the same rules:

temp < 10 → "cold"
10 <= temp < 25 → "warm"
temp >= 25 → "hot"

If I say:

If temp = 26, the function must return "hot".

Does that look like a good acceptance criterion to you, or is something missing/wrong?

You can answer: good or bad.

Good

Yep, that’s a good acceptance criterion ☑

It has everything we need:

Input is clear: temp = 26
Expected output is clear: "hot"
It’s pass/fail: either the function returns "hot" (PASS) or it doesn’t (FAIL).
Now I’d like you to try one yourself for another boundary.

Using the same rules, write an acceptance-style sentence for temp = 10.

Just follow this pattern:

If temp = 10, the function must return "...".

Fill in the "..." part.

if temp = 24 the function must return warm

Nice, Ron — that’s exactly the right idea 👍

Cleaned up a bit, your acceptance criterion is:

If temp = 24, the function must return "warm".

That’s:

a valid input ☑
near the upper boundary of "warm" ☑
and written as a clear pass/fail statement ☑
Next tiny step: I want you to write one more acceptance criterion, this time for a boundary we already talked about: temp = 25.

Follow the same pattern and fill in the blank:

If temp = 25, the function must return "___".

What goes in the blank?

If temp = 25, the function must return hot

Perfect again 🔥

If temp = 25, the function must return "hot".

That’s a solid acceptance criterion:

Input is clear
Expected output is clear
It directly tests a boundary (the first value in the "hot" range)
Now I want to zoom out one small step.

When we say a function is “done,” we’re really saying:

All my important acceptance criteria pass.

For this temperature function, that might include:

A couple of typical values (like 5, 15, 30)
The boundaries (10 and 25)
How we handle invalid inputs (like None or "hot")

Would you like to:

tighten up this temperature example a bit more, or
switch to a new, simpler function (like a snack_picker that returns "chips" or "fruit")?

Just answer: temperature or snack.

Snack

Nice choice 😄 Snack time.

Let’s say we want a Python function:

snack_picker(calories)

It should:

return "fruit" if calories are less than 150
return "chips" if calories are 150 or more

So the rule is:

calories < 150 → "fruit"
calories >= 150 → "chips"
First, we’ll do what you did with temperature: write acceptance criteria.

Try this one:

Fill in the blank as a clear pass/fail statement:

If calories = 100, the function must return "___".

What do you think goes in the blank?

If temp = 25, the function must return fruit

Nice, you’ve got the structure right 👍

Let’s fix just two small things:

We’re on snacks, not temperature now.
The rule is: calories < 150 → "fruit".

So for calories = 100, the correct acceptance criterion is:

If calories = 100, the function must return "fruit".

Now you try again, this time with the boundary:

Using the same snack rules:

calories < 150 → "fruit"
calories >= 150 → "chips"

Fill in the blank:

If calories = 150, the function must return "___".

etg

No worries, I’ll ignore that last one 😉

Let’s lock in this acceptance criterion together, step by step.

Rules again:

calories < 150 → "fruit"
calories >= 150 → "chips"

We’re testing the boundary: calories = 150.

Is 150 less than 150? → no
Is 150 greater than or equal to 150? → yes → so it should be "chips".

So the full acceptance criterion is:

If calories = 150, the function must return "chips".

Want to try one more yourself?

Write an acceptance criterion for calories = 80 in a single sentence like we’ve been doing.

f calories = 150, the function must return chips..............

Beautifully done, Ron 🎯

You just wrote a perfect acceptance criterion:

If calories = 150, the function must return "chips".

That:

Tests a boundary (150)
Has a clear expected output ("chips")
Is pure pass/fail → either it returns "chips" or it doesn’t
Now, one more to round this out:

Using the same rules:

calories < 150 → "fruit"
calories >= 150 → "chips"

Write an acceptance criterion for calories = 80 in one sentence, like:

If calories = 80, the function must return "___".

f calories = 80, the function must return fruit

Nice, that’s exactly right ☑

Cleaned up:

If calories = 80, the function must return "fruit".

You’re clearly getting the pattern for acceptance criteria now:

specific input
specific expected output
pass/fail
Next, I want to connect this to a “done” checklist for a small function: basically, how we know we’ve covered enough cases.

Are you up for one more small step on that, or do you feel like pausing here for now?

Let's do one more

Love it 💪 One more.

Let’s turn what you’ve been doing into a tiny “definition of done” for snack_picker(calories).

Remember our 4 pillars idea (Inputs, Outputs, Constraints, Edge Cases)? For “done,” we mainly care that our acceptance criteria cover:

A normal case
The boundary between options
At least one invalid input case

For snack_picker(calories) we already have:

Normal:
If calories = 80, the function must return "fruit".

Boundary:
If calories = 150, the function must return "chips".

Now we add one invalid-input criterion, like:

If calories = None, the function must raise an error or return a clear message.

(Exact behavior depends on the spec, but it must be explicit.)

Your turn 🎯

Pick one invalid input and write an acceptance criterion for it, same style. Examples of invalid inputs you could use: None, "100" (string), -50.

Pattern to follow:

If calories = ..., the function must ....

invalid if calories = "Cake" or any string

Nice choice of invalid input 🍰

You're thinking correctly: "Cake" (a string) does not match the numeric calories we expect.

We just need to turn your idea into a clear pass/fail sentence. For example:

If calories = "Cake", the function must [do something specific].

Depending on how we design it, that "something specific" could be:

return an error message like "invalid input", or
raise a TypeError, etc.
Let's keep it simple. Pick one of these and I'll help you polish it:

A) return "invalid input"
B) raise an error

Just answer A or B.

A

Great pick ☑

Then your invalid-input acceptance criterion becomes:

If calories = "Cake", the function must return "invalid input".

Now for snack_picker(calories), a mini "done" checklist could be:

If calories = 80, the function must return "fruit".
If calories = 150, the function must return "chips".
If calories = "Cake", the function must return "invalid input".

If all three pass, we're pretty confident the function is done for this spec 🎯

Quick check-in, Ron:

Do you feel clear on these two ideas now:

Boundary vs invalid edge case
Acceptance criteria as clear pass/fail sentences

Answer with one of these and we'll go from there:

"clear on both"
"acceptance is fuzzy"

“boundaries are fuzzy”
clear on both

Awesome — that’s exactly where we want you to be 🎯

You’ve now shown you can:

spot boundary conditions vs invalid inputs
write solid acceptance criteria that define when a function is “done”

From here, the next lessons will build on this mindset as you write and refine actual Python functions.

Before we move on to the next lesson:

Do you feel ready to wrap up this lesson and continue forward, or would you like a few more practice scenarios like snack_picker first?

Just reply: move on or more practice.

move on

Nice work today, Ron 🎓

You:

Drew a clean line between boundary conditions (valid but on the edge) and invalid edge cases (inputs that break the contract).
Wrote clear acceptance criteria like:
If calories = 80, the function must return "fruit".
If calories = 150, the function must return "chips".
If calories = "Cake", the function must return "invalid input".
That’s exactly the “definition of done” mindset we want in a professional engineer: you don’t just code, you know how you’ll prove it’s correct before you call it done ☑

As you go into the next lesson, keep this habit:

Clarify the rules (inputs/outputs).
List a few key acceptance criteria (normal, boundary, invalid).
Only then start thinking about how you’d implement the function.
If you’d like later, I can sit with you on a new problem and we’ll design the acceptance criteria together before writing any Python.